Understanding Quantization

From Scales and Zero-Points to INT4, FP8,
and Modern LLM Inference

A short technical handbook for AI engineers, developers, and serious learners

@techNmak

- ORIENTATION

What this handbook will make clear

Quantization is often summarized as "use fewer bits." That is directionally true and technically incomplete. A useful explanation has to say what tensor is quantized, what numerical codebook is used, how scale parameters are chosen, what is rounded or clipped, which operations actually run at low precision, and which costs remain at higher precision.

By the end, you should be able to explain:

- the affine quantize/dequantize equations and the roles of scale and zero-point;
- symmetric vs asymmetric quantization and per-tensor vs per-channel/per-group granularity;
- why rounding, clipping, calibration, and outliers create different error modes;
- weight-only, weight-activation, and KV-cache quantization as different deployment problems;
- PTQ, QAT, fake quantization, and dynamic vs static activation quantization;
- why "4-bit model" does not mean every value or every arithmetic operation is 4-bit;
- how LLM.int8(), SmoothQuant, GPTQ, AWQ, NF4/QLoRA, FP8, KIVI, and KVQuant attack different bottlenecks;
- why lower bit-width can reduce memory without automatically reducing wall-clock latency;
- how hardware kernels, packing, group size, scales, and mixed precision determine the real system outcome.

Prerequisites

Comfort with vectors, matrices, Transformer blocks, and the difference between weights, activations, and KV cache is enough. The earlier handbooks on KV cache and the Transformer block are useful context.

Scope

This handbook is about numerical representation and execution. It does not become a CUDA kernel manual, a complete mixed-precision training guide, or a benchmark catalog. Hardware-specific behavior is included only where it changes the mental model.

Research anchors: [1]-[17]

1 - WHAT QUANTIZATION IS

Replace a large numerical set with a smaller representable set

A model normally stores or computes values in some finite numerical format. Quantization maps those values to a smaller set of representable levels. A simple scalar quantizer can be written as

$$q = Q(x), \qquad \hat{x} = D(q),$$

where q is the stored low-precision code and $\hat{x}$ is the value reconstructed for later computation.

For a neural network, x might be a weight, activation, gradient, optimizer state, or KV-cache element. These objects have different distributions and different sensitivity to error, so "the model is quantized" is not a complete specification.

A useful deployment description therefore needs at least four pieces:

1. **target:** weights, activations, KV cache, or another tensor;
2. **format:** INT8, INT4, FP8, NF4, FP4, or another codebook;
3. **granularity:** one scale for a tensor, channel, token, group, or block;
4. **execution:** how packed values are consumed by the actual kernel.

Quantization is lossy in the ordinary case because multiple high-precision values map to the same code. The engineering goal is not to preserve every number exactly. It is to preserve the model behavior that matters while reducing memory, bandwidth, or compute cost.

Mental model. Bit-width is only one dimension of a quantization scheme. The mapping, scale granularity, calibration rule, and kernel are equally important.

Research anchors: [1], [2]

2 - LOW PRECISION IS NOT ONE THING

Integer quantization, floating-point formats, and codebooks differ

The phrase "8-bit" can describe numerically very different representations.

Example	Representation idea	Spacing of values
INT8 affine	integer code plus scale/zero-point	uniform under a fixed scale
FP8	sign, exponent, mantissa	non-uniform; dynamic range comes from exponent
NF4	4-bit index into a fixed non-uniform codebook	non-uniform quantile-style levels
BF16/FP16	reduced floating point	floating-point spacing, usually treated as mixed precision rather than PTQ

This distinction matters because two "8-bit" methods can have different dynamic range, error distribution, metadata, hardware support, and arithmetic paths.

BF16 and FP16 are usually discussed as reduced-precision floating-point formats rather than post-training quantization schemes. FP8 sits closer to the boundary: modern training/inference systems use scaling recipes around FP8 tensors, so it is often discussed in the same low-precision toolbox as quantization.

Likewise, NF4 is not simply "INT4 with a different scale." It is a 4-bit codebook designed around a normalized normal-weight assumption in QLoRA. Current bitsandbytes code explicitly treats NF4 as a lookup codebook rather than a sign/exponent/mantissa float format.

Important. Never infer the numerical behavior of a format from bit-width alone. "4-bit" or "8-bit" is not enough to reconstruct the quantizer.

Research anchors: [8], [9], [14], [16]

3 - AFFINE INTEGER QUANTIZATION

Scale plus zero-point map real values to integer codes

A common uniform affine quantizer represents a real value approximately as

$$x \approx s(q - z),$$

where $s > 0$ is a scale, z is an integer zero-point, and q is an integer code in $[q_{\min}, q_{\max}]$.

Quantization and dequantization are then

$$q = \text{clamp}\Big(\text{round}\Big(\frac{x}{s}\Big) + z,\ q_{\min}, q_{\max}\Big),$$

$$\hat{x} = s(q - z).$$

The scale determines the distance between adjacent representable real values. The zero-point determines which integer code represents real zero. Rounding produces finite precision; clamping handles values outside the chosen representable range.

For an asymmetric min/max mapping, a common starting point is

$$s = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}, \qquad z \approx q_{\min} - \frac{r_{\min}}{s},$$

with z rounded and clipped to the integer range.

The exact conventions vary by runtime. Some symmetric INT8 implementations intentionally use $[-127, 127]$ instead of the full signed $[-128, 127]$ range to preserve exact symmetry around zero.

Mental model. Affine quantization is a coordinate transform between a real-valued interval and a finite integer grid.

Research anchors: [1], [2], [13]

4 - SYMMETRIC VS ASYMMETRIC

Do we need a learned zero-point?

In **symmetric** quantization, the real range is centered around zero. For signed integers, the zero-point is usually fixed at zero and the scale is determined by the largest absolute magnitude:

$$s = \frac{\max |x|}{q_{\max}}, \qquad z = 0.$$

This is simple and often attractive for weights whose distributions are approximately centered.

In **asymmetric** quantization, the observed minimum and maximum can map to different ends of the integer range. The zero-point shifts the grid so that real zero is still representable. This can use the available code range more efficiently when the tensor is strongly offset from zero.

	Symmetric	**Asymmetric**
Zero-point	typically fixed at 0	derived from range
Best intuition	centered range	shifted range
Metadata	simpler	includes nonzero z
Common use	many weight schemes	activations / KV schemes where offset matters

There is no universal rule that weights must be symmetric or activations asymmetric. The choice depends on the distribution, backend, and quantization method.

Important. A nonzero zero-point is not "bias." It is part of the numerical coordinate system used to decode integer codes back into approximate real values.

Research anchors: [1], [2], [13]

5 - WORKED EXAMPLE I: SYMMETRIC INT8

Quantize, store, then reconstruct

Take

$$x = [-1.2, -0.5, 0, 0.3, 1.0]$$

and use signed symmetric INT8 with codes $[-127, 127]$.

The largest magnitude is 1.2, so

$$s = \frac{1.2}{127} \approx 0.00944882.$$

With $z = 0$,

$$q = \text{round}(x/s) = [-127, -53, 0, 32, 106].$$

Dequantization gives

$$\hat{x} = sq \approx [-1.2000, -0.5008, 0, 0.3024, 1.0016].$$

The reconstruction error (x_hat - x) is therefore approximately

$$[0, -0.0008, 0, +0.0024, +0.0016].$$

Three things are visible in this tiny example. First, the endpoints can be represented almost exactly when the scale is derived from the maximum magnitude. Second, interior values move to the nearest grid point. Third, with round-to-nearest, an unclipped scalar is displaced by at most roughly half one quantization step (apart from tie-breaking conventions).

If an outlier forced the scale to become much larger, the grid would become coarser for every smaller value sharing that scale. That is one reason granularity and outlier handling matter so much in LLM quantization.

Mental model. Quantization error is not mysterious: it is the distance between the original value and the nearest allowed reconstruction level, plus any clipping error.

Research anchors: [1], [2]

6 - WORKED EXAMPLE II: ASYMMETRIC UINT8

A shifted range uses a nonzero zero-point

Take

$$x = [-0.4, 0.24, 0.96, 1.6]$$

and map the observed range to UINT8 codes $[0, 255]$.

The range width is 2.0, so

$$s = \frac{1.6 - (-0.4)}{255} = \frac{2}{255} \approx 0.00784314.$$

The zero-point is

$$z \approx 0 - \frac{-0.4}{s} = 51.$$

Quantization gives

$$q = \text{round}(x/s + 51) = [0, 82, 173, 255].$$

Reconstruction gives

$$\hat{x} = s(q - 51) \approx [-0.4000, 0.2431, 0.9569, 1.6000].$$

The nonzero zero-point lets integer code 51 represent real zero even though the observed range is not symmetric around zero. That is the central reason asymmetric affine quantization exists.

A runtime may choose a range using calibration statistics rather than the exact minimum and maximum. Clipping a small number of extreme values can sometimes reduce total error because it allows a finer grid over the region where most values live.

Important. "Use min and max" is a simple calibration rule, not a law. Practical quantizers may optimize clipping thresholds or use histograms, percentiles, learned ranges, or method-specific objectives.

Research anchors: [1], [12], [13]

7 - ROUNDING, CLIPPING, AND ERROR

Three different sources of distortion

Quantization error is often discussed as one thing, but at least three mechanisms should be separated.

Rounding error. A value inside the representable interval is moved to the nearest grid/codebook level.

Clipping error. A value outside the selected range is saturated to an endpoint. This can be much larger than one quantization step.

Scale-sharing error. A group of heterogeneous values shares one scale. A large outlier can force a coarse step size for many otherwise small values.

This creates a central trade-off. A wider representable range reduces clipping but increases the spacing between levels. A narrower range improves resolution for common values but clips more extremes.

For uniform quantization, the approximate step size is proportional to

$$\Delta \propto \frac{\text{range width}}{\text{number of codes}}.$$

Halving bit-width dramatically reduces the number of levels: 8 bits gives 256 codes, while 4 bits gives only 16. At low bit-width, scale selection and distribution shape therefore matter much more.

Methods such as GPTQ, AWQ, SmoothQuant, and OmniQuant differ partly in how they reduce the downstream effect of these errors rather than merely applying naive round-to-nearest everywhere.

Mental model. Low-bit quantization is an error-allocation problem: decide where precision matters and where approximation can be tolerated.

Research anchors: [5], [6], [7], [12]

8 - QUANTIZATION GRANULARITY

One scale for how many values?

A quantizer can share parameters across different scopes.

Granularity	One scale covers	Trade-off
Per-tensor	entire tensor	least metadata, weakest adaptation
Per-channel	one channel/row/column	better fit, more scales
Per-group	fixed-size group of weights	common compromise in low-bit LLM weights
Per-token	values associated with one token	adapts to token-varying activations/cache
Per-block	hardware-defined block	enables local scaling and hardware-friendly formats

The axis matters. "Per-channel" is incomplete unless we say which dimension defines the channel. Weight matrices can be grouped along input or output dimensions depending on the method/kernel. KV-cache methods can choose different granularities for keys and values.

Finer granularity usually reduces quantization error because each scale covers a more homogeneous subset. But it also increases scale metadata and can complicate packing, vectorization, or kernel design.

This is why group size is not just an accuracy knob. It is also a systems parameter.

Important. Two models both described as "INT4" can behave differently because one uses per-group scales of 32 weights and another uses 128, or because one stores zero-points while the other is symmetric.

Research anchors: [2], [5], [6], [10], [11], [14]-[16]

9 - WORKED EXAMPLE III: GRANULARITY

Per-channel scales can protect small channels from large ones

Consider a two-row weight matrix

$$W = \begin{bmatrix} -0.9 & -0.2 & 0.7 \\ -12 & 3 & 8 \end{bmatrix}.$$

With one symmetric INT8 scale for the whole tensor,

$$s_{\text{tensor}} = 12/127 \approx 0.09449.$$

The first row is reconstructed approximately as

$$[-0.9449, -0.1890, 0.6614].$$

Its relatively small values are forced onto a grid determined by the 12 in the second row.

With one scale per row,

$$s_1 = 0.9/127 \approx 0.00709, \qquad s_2 = 12/127 \approx 0.09449.$$

The first row becomes approximately

$$[-0.9000, -0.1984, 0.7016].$$

For this toy matrix, mean-squared reconstruction error drops from about

$$8.62 \times 10^{-4} \quad \text{to} \quad 2.59 \times 10^{-4}.$$

The exact numbers are not a universal claim about per-channel quantization. They show the mechanism: separating groups prevents one high-dynamic-range channel from setting the precision of another.

Mental model. Finer granularity spends more metadata to buy locally better resolution.

Research anchors: [1], [2], [13]

10 - WHAT CAN BE QUANTIZED?

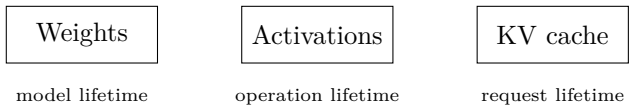
Weights, activations, and KV cache have different lifetimes

Three important LLM targets are easy to conflate.

Weights are model parameters reused across requests. Weight-only quantization primarily reduces model storage and weight bandwidth.

Activations are intermediate tensors produced while executing the model. Their distribution changes with input, layer, token, and execution phase. Activation quantization can enable low-precision matrix multiplication, but it must handle runtime variation and outliers.

KV cache is persistent per-request attention state during autoregressive inference. Its memory grows with cached sequence length, batch/concurrency, layers, KV heads, head dimension, and bytes per element.



A W4A16 model, a W8A8 model, and an FP8-KV-cache model solve different resource problems even if all are called "quantized LLMs."

Important. Always write the quantization target explicitly. W4A16 means 4-bit weights with 16-bit activations; accumulator precision is separately defined by the kernel. It is not the same system as W4A8 or 4-bit KV cache.

Research anchors: [3]-[11], [15]

11 - CALIBRATION AND OBSERVERS

How do we choose scales before deployment?

Post-training quantization often needs representative data to estimate ranges, clipping thresholds, activation statistics, or reconstruction objectives. This process is usually called **calibration**.

A simple observer may track running minimum and maximum. Histogram-based observers can estimate a distribution and choose a range using a more elaborate criterion. Per-channel observers collect separate statistics for different channels.

PyTorch's legacy torch.ao.quantization APIs still expose min/max, moving-average, per-channel, and histogram observers. Current quantization development is centralized in torchao; the exact calibration strategy used by an LLM method can be more specialized.

Calibration data is not necessarily the original training data. GPTQ, AWQ, SmoothQuant, and OmniQuant all use small calibration sets/statistics in different ways:

- GPTQ uses calibration inputs in a reconstruction procedure informed by approximate second-order information;
- AWQ uses activation statistics to identify/protect salient weight channels;
- SmoothQuant uses activation magnitudes to compute an equivalent rescaling between activations and weights;
- OmniQuant optimizes clipping and equivalent transformations on calibration samples.

Poor calibration can produce a technically valid quantized model whose scales are mismatched to real workloads.

Mental model. Calibration is not "testing accuracy." It is the process of estimating quantizer parameters from representative numerical behavior.

Research anchors: [5]-[7], [12], [13]

12 - STATIC VS DYNAMIC PTQ

When are activation quantization parameters chosen?

Classical quantization terminology distinguishes two common post-training patterns.

Static PTQ uses a calibration pass to determine activation quantization parameters before deployment. At inference, the runtime reuses those parameters.

Dynamic quantization typically stores quantized weights ahead of time but derives activation scales from the current runtime tensor. This adapts to input variation but adds runtime work.

LLM literature sometimes uses different method-specific language, so these terms should not be mechanically applied to every modern kernel. Weight-only GPTQ/AWQ models, FP8 scaling recipes, and KV-cache quantizers each have their own scale-update rules.

The core systems trade-off is still useful:

$$\text{precomputed statistics} \quad \leftrightarrow \quad \text{runtime adaptation.}$$

Runtime adaptation can improve fit to the current tensor, but scale calculation, reduction operations, conversions, and synchronization can erase part of the performance benefit if kernels are not designed around them.

Important. "Dynamic" does not mean the weights themselves are retrained or changed. It refers to when quantization parameters for runtime values are determined.

Research anchors: [1], [13], [14]

13 - QUANTIZATION-AWARE TRAINING

Train while simulating the quantizer

Post-training quantization modifies a trained model. Quantization-aware training (QAT) instead exposes the optimization process to quantization effects.

A typical QAT forward pass uses **fake quantization**: values are quantized to discrete codes and immediately dequantized back to floating point so the network experiences rounding/clipping behavior while training machinery can still update floating-point parameters.

Conceptually,

$$x \to q = Q(x) \to \hat{x} = D(q) \to \text{rest of network.}$$

The round operation is not ordinarily differentiable, so practical QAT uses surrogate gradient behavior such as straight-through estimators or framework-specific fake-quant modules.

QAT can recover accuracy at difficult bit-widths because the model learns around the quantizer. The cost is that it requires a training/fine-tuning pipeline, data, and compute. Large-model deployment therefore often prefers PTQ when quality remains acceptable.

Jacob et al. combined quantization with training to enable integer-arithmetic-only inference, while current PyTorch APIs expose fake-quantization and observer modules for QAT workflows.

Mental model. PTQ asks "how well can this trained model survive a quantizer?" QAT asks "can training adapt the model to the quantizer?"

Research anchors: [1], [13]

14 - INTEGER-ONLY VS DEQUANTIZED EXECUTION

Storage format does not uniquely determine arithmetic format

A quantized tensor can participate in several execution paths.

In an integer pipeline, inputs and weights may be represented as integers, matrix multiplication accumulates into a wider integer type, and scale factors convert the result to the next numerical domain. Jacob et al. developed this style for integer-arithmetic-only inference.

In many LLM weight-only kernels, packed low-bit weights are read from memory, unpacked/dequantized inside the kernel, and multiplied with higher-precision activations. The memory traffic is low-bit even though the arithmetic path is not "INT4 everywhere."

Modern hardware can also provide native low-precision matrix engines for INT8, FP8, or FP4-style formats. Which path is fastest depends on device support, tensor shape, packing, kernel fusion, and whether conversion overhead is hidden.



Important. A checkpoint's storage bit-width, the kernel input bit-width, the multiplier format, and the accumulator format can all be different.

Research anchors: [1], [4], [9], [14], [15]

15 - WEIGHT-ONLY QUANTIZATION

Compress the model without forcing activations low

Weight-only quantization is attractive for autoregressive LLM inference because model weights are large and repeatedly streamed through matrix multiplications, especially at small decode batch sizes.

A notation such as W4A16 usually means the weights are stored in a 4-bit representation while activations remain around FP16/BF16 precision. The kernel can load packed 4-bit weights, dequantize/scaled them on chip, and combine them with higher-precision activations.

Benefits can include:

- lower checkpoint and device-memory footprint;
- lower weight bandwidth per token;
- larger models fitting on a fixed device;
- potential latency/throughput gains when the kernel is bandwidth-bound and optimized.

But weight-only quantization does not shrink activation memory, KV-cache memory, or every intermediate tensor. It also does not guarantee a proportional compute speedup.

GPTQ and AWQ are two prominent low-bit weight-only families. They differ in how they choose the quantized weights/scales, but both rely on specialized kernels to convert compression into deployment performance.

Mental model. Weight-only quantization attacks the bytes used to represent and move model parameters. It does not quantize the entire inference state.

Research anchors: [6], [7], [15]

16 - WHY “4-BIT MODEL” IS NOT 4-BIT COMPUTE

Storage, metadata, and accumulation survive the slogan

Suppose a model has nominally 4-bit weights. Several higher-precision objects can still remain:

- per-group scales and possibly zero-points;
- biases and normalization parameters;
- embeddings or selected layers kept at higher precision;
- activations in FP16/BF16;
- accumulators in INT32, FP16, BF16, or FP32 depending on the kernel;
- KV cache at another dtype;
- temporary dequantized fragments inside the kernel.

QLoRA is a clean example of why this language matters. It stores the frozen base model in a 4-bit NF4 representation, but computation for training uses a higher-precision compute dtype and gradients flow into LoRA adapters rather than turning the whole training process into 4-bit arithmetic.

Likewise, W4A16 TensorRT-LLM recipes explicitly name different precisions for weights and activations.

So the phrase “4-bit model” is best read as a shorthand for a particular storage/quantization contract, not as a claim that every number in memory and every multiply is four bits wide.

Important. When comparing quantized systems, ask for the full precision tuple: weights, activations, accumulators, KV cache, scales, and any unquantized exceptions.

Research anchors: [8], [15], [16]

17 - ACTIVATION OUTLIERS

A few large dimensions can set the scale for many ordinary values

LLM activations can contain persistent high-magnitude features. If one scale is shared across a tensor or vector, these outliers expand the required dynamic range and make the quantization step coarser for the majority of smaller values.

This is why activation quantization can be harder than weight quantization. The activation distribution also changes with input and runtime state, while weights are fixed after training.

Several influential methods respond differently:

- **LLM.int8()** isolates outlier feature dimensions and computes them in higher precision while most values use INT8;
- **SmoothQuant** rescales activations and weights with an equivalent transformation so activation outliers become easier to quantize;
- **AWQ** uses activation statistics to identify weight channels whose quantization error matters most, then rescales to protect them.

These are not interchangeable algorithms. They share a lesson: the numerical distribution seen by the matrix multiply matters more than nominal bit-width alone.

Mental model. Outliers are expensive because shared scales turn a local extreme into a global loss of resolution.

Research anchors: [3], [5], [7]

18 - LLM.INT8()

Keep almost all matrix multiplication in INT8, isolate outliers

LLM.int8() was designed for large Transformer inference where naive INT8 quantization degraded quality as model scale increased.

The method combines two ideas. First, most values use vector-wise INT8 quantization with separate normalization constants for inner products. Second, feature dimensions identified as outliers are separated and processed in higher precision. The paper reports that more than 99.9% of values can still participate in the 8-bit path while outlier dimensions use a 16-bit matrix multiplication.

This is a mixed-precision decomposition, not a claim that the whole model is pure INT8 arithmetic.

The systems results are also a useful warning against simplistic speed claims. The paper's supplemental benchmarks show that quantization/dequantization overhead can dominate smaller matrix multiplications, while larger model dimensions benefit more from INT8 execution.

Important. The lesson from LLM.int8() is broader than one algorithm: low-bit arithmetic only helps when numerical exceptions and conversion overhead are handled in a hardware-aware way.

Research anchors: [3]

19 - SMOOTHQUANT

Move quantization difficulty from activations into weights

SmoothQuant targets W8A8 post-training quantization for LLM matrix multiplications. Its core observation is that weights are typically easier to quantize than activations with strong outliers.

For an activation channel X_j and corresponding weight channel W_j, SmoothQuant introduces a per-channel scaling factor s_j and applies an algebraically equivalent transformation of the form

$$XW = (X\ \mathrm{diag}(s)^{-1})\,(\mathrm{diag}(s)W).$$

The full-precision product is unchanged before quantization. Numerically, however, activation magnitudes can be reduced while weight magnitudes grow, transferring part of the quantization difficulty from activations to weights.

The smoothing strength is controlled by method-specific statistics/hyperparameters. The transform is computed offline from calibration data, so deployment does not require retraining the original model.

SmoothQuant's published speedups are tied to its INT8 kernels, models, and hardware. The durable concept is the equivalent rescaling, not a universal latency multiplier.

Mental model. Sometimes the best quantizer changes the representation before rounding rather than accepting the original tensor distribution.

Research anchors: [5]

20 - ZEROQUANT

Quantization is also a backend problem

ZeroQuant is useful because it makes an often-missed point explicit: a quantization algorithm and an efficient quantized system are different layers of the solution.

The method combines fine-grained weight/activation quantization with layer-wise knowledge distillation options and an optimized backend intended to remove quantization/dequantization overhead. In the paper's experiments, INT8 weight-activation inference and mixed INT4/INT8 schemes achieved model- and hardware-specific efficiency improvements relative to FP16.

The exact benchmark numbers are less important than the architecture of the solution:

1. choose quantization granularity/ranges;
2. recover quality if needed;
3. implement kernels that keep conversion overhead from erasing the savings.

A mathematically good low-bit representation can still be slow if the runtime repeatedly materializes dequantized weights in global memory, launches extra kernels, or falls back to unsupported operators.

Mental model. Quantization has an algorithmic half and a systems half. Compression without an execution path is not an inference optimization.

Research anchors: [4]

21 - GPTQ

One-shot weight quantization using approximate second-order information

GPTQ is a post-training weight quantization method designed to compress very large generative Transformers to low bit-widths without full retraining.

At a high level, it quantizes weights layer by layer while compensating for the error introduced by earlier quantized weights. The update uses approximate second-order information derived from calibration activations, following the Optimal Brain Quantization lineage but reorganized to make the procedure practical at LLM scale.

This is different from simply rounding every weight independently. GPTQ tries to minimize the effect of quantization on the layer's outputs under representative inputs.

The original paper demonstrated 3- and 4-bit weight quantization for very large GPT/OPT-style models and included custom low-bit kernels. Modern implementations add details such as group size, activation ordering, static groups, packing layout, and backend-specific kernels.

Important. "GPTQ 4-bit" is not one universal binary format. Accuracy and runtime depend on group size, symmetric/asymmetric choices, kernel implementation, and calibration configuration.

Research anchors: [6]

22 - AWQ

Use activations to identify which weight channels matter most

AWQ is another post-training, low-bit weight-only method, but its mechanism differs from GPTQ.

The paper observes that quantization error is not equally important across weight channels. Activation statistics are used to identify salient channels, and an equivalent per-channel scaling transformation reduces the error on important weights without storing a mixed-precision set of protected weights.

AWQ deliberately avoids backpropagation and layer reconstruction. Its calibration process searches scaling factors from activation statistics, then the resulting weights are packed for hardware-friendly inference.

This gives a useful conceptual contrast:

- GPTQ uses a reconstruction/second-order error-compensation procedure;
- AWQ uses activation-aware scaling to protect salient weight channels;
- both are commonly deployed as low-bit weight-only quantization with specialized kernels.

Published TinyChat speedups are specific to the paper's devices and implementation, not a property of AWQ bit-width in isolation.

Mental model. The quantization method decides which errors to avoid; the kernel decides whether the compressed representation becomes a performance win.

Research anchors: [7]

23 - GPTQ, AWQ, SMOOTHQUANT, LLM.INT8

Similar vocabulary, different problems

Method	Main target	Core idea	Typical deployment framing
LLM.int8()	W/A matmul	isolate outlier dimensions	mixed INT8/16-bit
SmoothQuant	W8A8	migrate activation difficulty to weights	INT8 matmul
GPTQ	weights	reconstruction with second-order information	W3/W4A16-style
AWQ	weights	activation-aware channel scaling	W4A16-style

The table is intentionally conceptual. Real frameworks support multiple variants and bit-widths.

Three distinctions prevent confusion:

1. **weight-only vs weight-activation:** reducing model bandwidth is different from enabling low-bit activation matmuls;
2. **algorithm vs format:** GPTQ and AWQ are methods, not primitive numerical data types;
3. **quantizer vs runtime:** the same quantized checkpoint can perform differently across kernels and hardware.

Important. Do not ask "is GPTQ better than INT4?" GPTQ is a way to produce low-bit weights; INT4 describes a numerical/storage family. The comparison mixes levels of abstraction.

Research anchors: [3], [5]-[7]

24 - NF4

A 4-bit codebook for normally distributed weights

QLoRA introduced NormalFloat 4 (NF4) for quantizing pretrained weights that are approximately normally distributed after normalization by a local absolute maximum.

NF4 uses 16 representable codebook values chosen from quantiles of a normal distribution and normalized into a bounded range. A stored 4-bit value is therefore an index into this codebook. It is not a conventional floating-point encoding with sign, exponent, and mantissa fields.

Current bitsandbytes source makes this explicit: NF4 and FP4 use different codebooks/encodings, even though both consume four bits per stored index.

QLoRA describes NF4 as information-theoretically optimal for normally distributed weights under its stated construction. A later technical note argues that strict optimality does not carry through finite-block absmax normalization because the post-normalization distribution depends on block size. Treat the original claim as method-specific motivation, not a universal property. [8,17]

non-uniform codebook levels (schematic)

Mental model. NF4 compresses by choosing a non-uniform set of 16 reconstruction values tailored to a weight-distribution assumption.

Research anchors: [8], [16], [17]

25 - QLORA AND DOUBLE QUANTIZATION

Quantize the frozen base model, train small adapters

QLoRA combines low-bit storage with parameter-efficient fine-tuning. The pretrained base model is frozen and stored in 4-bit NF4, while gradients are backpropagated through its dequantized computations into trainable LoRA adapters.

Two additional ideas matter:

Double quantization quantizes the quantization constants themselves, reducing scale metadata overhead.

Paged optimizers use unified-memory paging to manage transient optimizer-memory spikes during fine-tuning.

The compute dtype remains higher precision than the 4-bit storage. This is why describing QLoRA as "training in 4-bit" without qualification is misleading.

QLoRA's contribution is also a useful reminder that quantization can be used to make *training/fine-tuning memory* feasible, not only inference deployment.

Important. NF4 belongs to the storage/quantization representation of the frozen base weights. LoRA adapter parameters, optimizer states, gradients, activations, and compute dtypes follow their own precision rules.

Research anchors: [8], [16]

26 - FP8: E4M3 AND E5M2

Floating-point quantization trades precision for dynamic range

The widely used FP8 proposal defines two 8-bit floating-point formats:

- **E4M3**: 1 sign bit, 4 exponent bits, 3 mantissa bits;
- **E5M2**: 1 sign bit, 5 exponent bits, 2 mantissa bits.

E5M2 has wider dynamic range and less significand precision. E4M3 has more precision within a narrower range.

Unlike affine INT8 under one scale, FP8 values are not uniformly spaced in real-number space. The exponent changes the step size with magnitude.

The original FP8 work demonstrated training and inference across neural architectures, including large language models, using scaling strategies around these formats. NVIDIA Transformer Engine uses E4M3/E5M2 recipes on supported GPUs and commonly keeps sensitive operations at higher precision.

This is why "FP8 model" should not be read as "every operation executes in FP8." Layer normalization, softmax, reductions, accumulations, and unsupported operations can remain BF16/FP16/FP32 depending on the recipe.

Mental model. Integer quantization uses a scaled uniform grid; FP8 uses a floating-point grid whose spacing changes with exponent.

Research anchors: [9], [14]

27 - FP8 SCALING AND MIXED PRECISION

The format is only half of an FP8 recipe

An FP8 tensor has limited dynamic range, so practical systems maintain scale information that maps a higher-precision tensor into the available FP8 range.

Transformer Engine exposes several strategies, including delayed scaling, current per-tensor scaling, and blockwise variants. A typical scale derives from an observed absolute maximum ($amax$) and the maximum representable FP8 value, with margins/history depending on the recipe.

Mixed precision remains essential. Current Transformer Engine documentation states that linear operations can run in low precision while normalization, softmax, and other operations remain at higher precision by default; attention precision also depends on recipe/backend configuration.

This separation explains why low-precision training can preserve stability: precision is spent differently across operations rather than forcing the entire graph into one format.

Important. A datatype name such as E4M3 does not fully specify an FP8 training/inference system. Scaling granularity, amax policy, accumulation precision, and operator coverage are part of the recipe.

Research anchors: [9], [14]

28 - BLOCK-SCALED FP8 AND FP4

Modern hardware moves scales closer to the data

Current implementation note (Transformer Engine 2.19, September 2026): recheck current docs.

Newer accelerator generations support finer-grained block scaling. Current NVIDIA Transformer Engine documentation, for example, describes MXFP8 on Blackwell-class hardware using local scaling for blocks of 32 values, and NVFP4 as a supported 4-bit low-precision recipe.

The reason is intuitive. One scale for an entire tensor must accommodate its global dynamic range. Giving each small block its own scale reduces the range each codebook has to cover, which can improve local precision.

This comes with additional metadata and layout rules. Blockwise formats can also require different quantized views for row-wise and column-wise use because quantizing a tensor and quantizing its transpose are not numerically identical under directional block grouping.

These formats are hardware- and library-specific enough that they should be version-scoped. The evergreen idea is:

$$\text{finer scale granularity} \Rightarrow \text{better local fit} + \text{more metadata/layout constraints.}$$

Important. NVFP4, NF4, and generic INT4 are three different numerical schemes. Sharing "4-bit" does not make them interchangeable.

Research anchors: [14], [15]

29 - KV-CACHE QUANTIZATION

Compress per-request state, not model weights

During autoregressive decoding, the KV cache can consume a large fraction of device memory for long contexts and high concurrency. Quantizing K/V state reduces bytes per cached token and can also reduce memory-bandwidth pressure when attention reads the cache.

For a conventional cache payload,

$$M_{KV} = 2LTH_{KV}D_h b,$$

where D_h is the per-head key/value dimension.

so reducing the storage bytes per element b directly reduces the idealized tensor payload.

But KV-cache quantization is numerically different from weight quantization. Keys and values are generated dynamically from each request. Their distributions can vary by layer, head, token, and model architecture. Quantization error also feeds future attention calculations for every subsequent decode step.

Modern runtimes therefore expose separate KV-cache quantization modes, including FP8 and newer low-precision formats on supported hardware.

Mental model. Weight quantization changes a shared model asset. KV-cache quantization changes growing, request-specific inference state.

Research anchors: [10], [11], [15]

30 - KIVI

Keys and values need not use the same granularity

KIVI studied the distributions of cached keys and values and found an asymmetry useful for very low-bit quantization.

The paper reports that keys exhibit channel-wise outlier structure, motivating **per-channel key quantization**, while values are better handled **per-token**. KIVI combines these choices in a tuning-free asymmetric 2-bit KV-cache method and retains a small residual region in higher precision for recent tokens in its practical implementation.

This is an important conceptual lesson: even tensors that appear as a pair in the attention equation can have different numerical statistics and therefore different best quantization axes.

KIVI's reported memory and throughput improvements are tied to its models, workloads, kernel implementation, and experimental setup. The durable result for this handbook is the distribution-driven granularity choice.

Important. "Quantize K and V to 2-bit" is not a complete description of KIVI. The per-channel/per-token asymmetry is central to the method.

Research anchors: [10]

31 - KVQUANT

Long-context cache compression uses more than bit-width

KVQuant is another line of work on extremely low-bit KV-cache quantization for long-context inference. It studies quantization-aware choices such as per-channel key handling, non-uniform quantization, outlier treatment, and optimized kernels.

The broader point is that KV-cache compression at very low precision becomes a systems co-design problem:

- choose a numerical representation that preserves attention quality;
- decide what statistics/scales are stored per channel/token/group;
- handle outliers or recent tokens if needed;
- implement attention kernels that consume compressed K/V without expensive materialization.

A method can report a dramatic theoretical cache reduction and still fail to improve serving throughput if decompression becomes the bottleneck or the attention kernel cannot use the format efficiently.

KVQuant's published "10 million context" framing belongs to its research setup and should not be read as a universal context guarantee for arbitrary models or hardware.

Mental model. For KV cache, compression ratio, attention quality, and decode bandwidth must be evaluated together.

Research anchors: [11]

32 - MEMORY ACCOUNTING

Ideal bit-width savings are a lower bound on the full runtime footprint

For P parameters stored at b bits each, the ideal raw payload is

$$M_{\text{weights}} = \frac{Pb}{8} \text{ bytes.}$$

For an illustrative 7-billion-parameter model:

Weight payload	Decimal GB	GiB
16-bit	14.0	13.04
8-bit	7.0	6.52
4-bit	3.5	3.26

Real quantized checkpoints/runtime memory are larger because they can include scales, zero-points, alignment/padding, packing metadata, unquantized layers, biases/norms, codebook state, and temporary buffers.

Runtime memory also includes activations, KV cache, allocator overhead, CUDA graphs/-workspaces, and possibly multiple copies during loading/conversion.

Important. A 4-bit weight payload is approximately one quarter of a 16-bit weight payload. That does not imply total GPU memory use is one quarter.

Research anchors: [2], [6]-[8], [15], [16]

33 - WORKED MEMORY EXAMPLE: GROUP METADATA

A "4-bit" weight can cost more than 4 effective bits

Assume symmetric groupwise 4-bit weights with one FP16 scale for every 128 weights.

Each group stores:

$$128 \times 4 = 512 \text{ weight bits}$$

plus

$$16 \text{ scale bits.}$$

So the effective cost is

$$4 + \frac{16}{128} = 4.125 \text{ bits/weight.}$$

For 7 billion weights, that idealized payload becomes

$$7 \times 10^9 \times \frac{4.125}{8} = 3.609375 \times 10^9 \text{ bytes} \approx 3.36 \text{ GiB.}$$

If the scheme also stored one FP16 zero-point per group, the cost would become

$$4 + \frac{32}{128} = 4.25 \text{ bits/weight}$$

before alignment or other metadata.

This is still far below 16-bit storage, but it shows why "parameter count times half a byte" is only the first-order estimate.

Mental model. Group size controls both numerical locality and metadata amortization. Smaller groups usually improve fit while increasing scale/zero-point overhead.

Research anchors: [6]-[8], [16]

34 - KERNELS AND HARDWARE

Compression only becomes speed when the machine can consume it efficiently

Low-precision performance depends on the complete execution path.

A useful checklist is:

- Does the GPU/accelerator have native tensor-core/instruction support for the format?
- Are weights packed in the layout expected by the kernel?
- Are scales loaded efficiently and fused with dequantization?
- Is the operation GEMM-like or GEMV-like at the current batch/phase?
- Does the runtime fall back to a higher-precision implementation for unsupported shapes/operators?
- Is conversion performed on chip, or are dequantized tensors written back to global memory?

Current TensorRT-LLM exposes distinct recipes such as FP8, FP4, W4A16 GPTQ/AWQ, W4A8 variants, and quantized KV-cache modes. Transformer Engine similarly distinguishes FP8, MXFP8, and NVFP4 recipes with hardware constraints.

This is why a quantized checkpoint can be smaller yet slower on the wrong device or software stack.

Important. Bit-width is a property of representation. Speed is a property of representation + operator shapes + kernels + hardware + workload.

Research anchors: [14], [15]

35 - LATENCY, THROUGHPUT, AND BANDWIDTH

Lower precision changes different bottlenecks differently

Quantization can help several resources, but not always at the same time.

Capacity. Smaller weights/cache allow larger models or more concurrent requests to fit.

Bandwidth. Fewer bytes must be loaded for weight-bound decode or KV-bound attention.

Compute throughput. Native low-precision tensor cores can execute more operations per second on supported shapes.

Latency. End-to-end latency improves only if the reduced bottleneck dominates more than packing/dequantization/launch overhead.

Throughput. Memory savings can raise batch/concurrency even when single-request latency barely changes.

LLM.int8() provides a concrete historical example: smaller matrix multiplications could lose to quantization overhead, while larger dimensions benefited more. KIVI similarly shows how cache compression can increase feasible batch size, which changes throughput even if the arithmetic graph is otherwise similar.

Mental model. Ask which resource moved: bytes resident, bytes transferred, arithmetic throughput, batch capacity, or end-to-end time. “Faster” without a metric is incomplete.

Research anchors: [3], [10], [14], [15]

36 - A TRANSPARENT PYTORCH QUANTIZER

Pedagogical symmetric quantize-dequantize

The following code mirrors the worked INT8 example. It is intentionally simple and is not an optimized inference kernel.

```
import torch

def symmetric_qdq(x, bits=8):
    qmax = 2 ** (bits - 1) - 1
    scale = x.abs().max() / qmax
    scale = torch.clamp(scale, min=torch.finfo(x.dtype).eps)
    q = torch.clamp(
        torch.round(x / scale), -qmax, qmax
    ).to(torch.int8)
    x_hat = q.float() * scale
    return q, scale, x_hat

x = torch.tensor([-1.2, -0.5, 0.0, 0.3, 1.0])
q, scale, x_hat = symmetric_qdq(x)
# q = [-127, -53, 0, 32, 106]
# x_hat = [-1.2000, -0.5008, 0.0000, 0.3024, 1.0016]
```

For bits < 8, q still uses torch.int8 here; this example does not implement sub-byte packing.

A production implementation must additionally define granularity, dtype/range conventions, zero-points, packing layout, scale dtype, group size, clipping/calibration policy, operator support, and accumulation precision.

Important. This function demonstrates the numerical map. It does not make PyTorch execute the matrix multiplication with an optimized INT8 kernel merely because the tensor codes are integers.

Research anchors: [1], [13]

37 - COMMON IMPLEMENTATION MISTAKES

Most low-bit bugs are contract mismatches

1. **Comparing bit-widths without format.** NF4, FP4, INT4, and NVFP4 are not interchangeable.
2. **Dropping scale metadata from memory estimates.** Groupwise schemes need scales and sometimes zero-points.
3. **Using the wrong axis.** Per-channel along the wrong dimension can change both accuracy and kernel compatibility.
4. **Reusing bad calibration data.** Activation ranges can be workload-dependent.
5. **Assuming storage precision equals compute precision.** Weight-only kernels often compute/accumulate at higher precision.
6. **Ignoring unquantized exceptions.** Norms, embeddings, output heads, or selected layers may remain high precision.
7. **Benchmarking checkpoint size as latency.** Smaller files do not guarantee faster generation.
8. **Forgetting packing/layout.** A mathematically correct quantizer can be unusable by a target kernel.
9. **Mixing incompatible scales/checkpoints.** Quantization metadata is part of the model contract.
10. **Publishing paper speedups as universal.** Hardware, batch, sequence length, and software version must match.

Mental model. Treat a quantized checkpoint as weights + quantization metadata + a required execution contract.

Research anchors: [3]-[16]

38 - THE COMPLETE MENTAL MODEL

Quantization in ten questions

When you see a quantized model, ask:

1. **What is quantized?** Weights, activations, KV cache, or several of them?
2. **What format?** INT8, INT4, FP8, NF4, FP4, something else?
3. **What mapping?** Uniform affine, floating point, or non-uniform codebook?
4. **What granularity?** Tensor, channel, group, token, or block?
5. **How are scales chosen?** Calibration, runtime amax, reconstruction, learned clipping?
6. **What happens to outliers?** Clip, isolate, rescale, protect, or keep high precision?
7. **What actually computes low precision?** Storage and arithmetic may differ.
8. **What metadata remains?** Scales, zero-points, codebooks, exceptions, padding?
9. **Which bottleneck improves?** Capacity, bandwidth, compute, latency, throughput?
10. **Which kernel/hardware consumes it?** Without backend support, compression may not become speed.

GPTQ, AWQ, SmoothQuant, LLM.int8(), NF4, FP8, and KV-cache methods are easier to compare once these questions are explicit. They are not competing answers to one identical problem; they occupy different points in the design space.

Mental model. Quantization is controlled numerical approximation. The useful question is not "how many bits?" but "which information is approximated, with what map, and what system benefit does that approximation unlock?"

Research anchors: [1]-[17]

REFERENCES

Primary papers and current implementation sources

1. B. Jacob et al. *Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference.* CVPR 2018.
2. M. Nagel et al. *A White Paper on Neural Network Quantization.* arXiv:2106.08295, 2021.
3. T. Dettmers, M. Lewis, Y. Belkada, L. Zettlemoyer. *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale.* NeurIPS 2022. arXiv:2208.07339.
4. Z. Yao et al. *ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers.* NeurIPS 2022. arXiv:2206.01861.
5. G. Xiao et al. *SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models.* ICML 2023. arXiv:2211.10438.
6. E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh. *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers.* ICLR 2023. arXiv:2210.17323.
7. J. Lin et al. *AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration.* MLSys 2024. arXiv:2306.00978.
8. T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer. *QLoRA: Efficient Finetuning of Quantized LLMs.* NeurIPS 2023. arXiv:2305.14314.
9. P. Micikevicius et al. *FP8 Formats for Deep Learning.* arXiv:2209.05433, 2022.
10. Z. Liu et al. *KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache.* ICML 2024. arXiv:2402.02750.
11. C. Hooper et al. *KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization.* NeurIPS 2024. arXiv:2401.18079.
12. W. Shao et al. *OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models.* arXiv:2308.13137, 2023.
13. PyTorch and torchao documentation. Quantization observers, fake quantization, and current quantization/QAT APIs. Current documentation consulted September 2026.
14. NVIDIA Transformer Engine documentation. FP8, MXFP8, NVFP4, scaling recipes, mixed-precision execution. Current documentation consulted September 2026.
15. NVIDIA TensorRT-LLM documentation. Quantization recipes for FP8/FP4, GPTQ/AWQ, and KV cache. Current documentation consulted September 2026.
16. bitsandbytes documentation and source. NF4/FP4 codebooks, 4-bit packing, QLoRA integration. Current source consulted September 2026.
17. D. Yoshida. NF4 Isn't Information Theoretically Optimal (and that's Good). arXiv:2306.06965, 2023.